

A Novel Self-Supervised Framework Based on Masked Autoencoder for Traffic Classification

Ruijie Zhao^{1b}, *Graduate Student Member, IEEE*, Mingwei Zhan, Xianwen Deng^{1b}, *Student Member, IEEE*, Fangqi Li^{1b}, *Graduate Student Member, IEEE*, Yanhao Wang^{1b}, Yijun Wang^{1b}, Guan Gui^{1b}, *Senior Member, IEEE*, and Zhi Xue^{1b}

Abstract—Traffic classification is a critical task in network security and management. Recent research has demonstrated the effectiveness of the deep learning-based traffic classification method. However, the following limitations remain: (1) the traffic representation is simply generated from raw packet bytes, resulting in the absence of important information; (2) the model structure of directly applying deep learning algorithms does not take traffic characteristics into account; and (3) scenario-specific classifier training usually requires a labor-intensive and time-consuming process to label data. In this paper, we introduce a masked autoencoder (MAE) based traffic transformer with multi-level flow representation to tackle these problems. To model raw traffic data, we design a formatted traffic representation matrix with hierarchical flow information. After that, we develop an efficient Traffic Transformer, in which packet-level and flow-level attention mechanisms implement more efficient feature extraction with lower complexity. At last, we utilize MAE paradigm to pre-train our classifier with a large amount of unlabeled data, and perform fine-tuning with a few labeled data for a series of traffic classification tasks. Experiment findings reveal that our method outperforms state-of-the-art methods on five real-world traffic datasets by a large margin. The code is available at <https://github.com/NSSL-SJTU/YaTC>.

Index Terms—Network security, traffic classification, self-supervised learning.

I. INTRODUCTION

TRAFFIC classification is attracting the attention of service providers and equipment vendors as a significant solution for solving network management problems. Understanding

the content of traffic allows network operators to respond swiftly in support of diverse business goals, enhancing service quality and user experience [2], [3], [4], [5]. Meanwhile, traffic classification is a critical component of the intrusion detection system, which is utilized to detect threats and safeguard system security [6], [7], [8], [9]. However, the growing usage of encrypted traffic and anonymous network technology makes analyzing complicated traffic difficult. To construct a sophisticated traffic analyzer for more accurate traffic classification, it is necessary to capture implicit and robust patterns in diverse traffic.

Traditional traffic analysis methods identify different network services based on fundamental traffic features (e.g., communication protocol, port number), which are no longer appropriate owing to the complexity and variability of the current traffic [10]. To address this issue, several works use machine learning (ML) algorithms to classify traffic through statistical features [11], [12], [13]. However, these approaches rely on expert experience to select specific features, and even insignificant statistical features might impact analysis performance [14]. Deep learning (DL) techniques, which can automatically extract features from packet bytes of raw traffic, have been used by an increasing number of researchers for more effective traffic analysis in recent years. Unfortunately, once the length of a certain packet is too long, the bytes from that packet can overwhelm important information from other packets. Thus, the lack of representation design in traditional DL-based methods brings instability in performance. Besides, they often need a large amount of labeled traffic data for training. Labeling data is a time-consuming and labor-intensive operation, resulting in high deployment and update costs.

Pre-training methods with massive volumes of unlabeled data have recently demonstrated outstanding performance in computer vision (CV) [15], [16], [17], [18] and natural language processing (NLP) [19], [20] tasks. When compared to the traditional supervised learning paradigm, the self-supervised learning paradigm pre-training method has two major advantages: (1) more effective representations can reduce the reliance of downstream tasks on the amount of labeled data; and (2) better initialization parameters can speed up convergence and improve classification performance. Recent studies [21], [22] in traffic classification directly utilize the BERT-based paradigm [19], a pre-training method created for NLP tasks, and obtain better performance. However, unlike typical words, traffic bytes as input lack explicit high-level semantic units, making it unreasonable to build vocabularies to learn semantic relations. Furthermore, neither method designs

Manuscript received 31 March 2023; revised 7 October 2023; accepted 12 November 2023; approved by IEEE/ACM TRANSACTIONS ON NETWORKING Editor C. Wu. Date of publication 3 January 2024; date of current version 18 June 2024. This work was supported in part by the Shanghai Jiao Tong University (SJTU)-QI'ANXIN Joint Laboratory of Information System Security, in part by the National Key Research and Development Program of China under Grant 31020, in part by the NIO Student Innovation Funding Program, and in part by the Network Security & Ecosystem Innovation Laboratory of Intelligent Radio & Television Broadcasting of the National Radio and Television Administration under Grant TXX20200001ZSB001. Part of this work [DOI: 10.1609/aaai.v37i4.25674] has been accepted by the Thirty-Seventh AAAI Conference on Artificial Intelligence (AAAI 2023). (Corresponding authors: Yijun Wang; Zhi Xue.)

Ruijie Zhao, Mingwei Zhan, Xianwen Deng, Fangqi Li, Yijun Wang, and Zhi Xue are with the School of Electronic Information and Electrical Engineering, Shanghai Jiao Tong University, Shanghai 200240, China (e-mail: ruijiezhao@sjtu.edu.cn; mw.zhan@sjtu.edu.cn; 2594306528@sjtu.edu.cn; solour_lfq@sjtu.edu.cn; ericwyj@sjtu.edu.cn; zxue@sjtu.edu.cn).

Yanhao Wang is with the QI-ANXIN Technology Research Institute, Beijing 100015, China (e-mail: wangyanhao136@gmail.com).

Guan Gui is with the College of Telecommunications and Information Engineering, Nanjing University of Posts and Telecommunications, Nanjing 210003, China (e-mail: guiguang@njupt.edu.cn).

Digital Object Identifier 10.1109/TNET.2023.3335253

1558-2566 © 2024 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See <https://www.ieee.org/publications/rights/index.html> for more information.

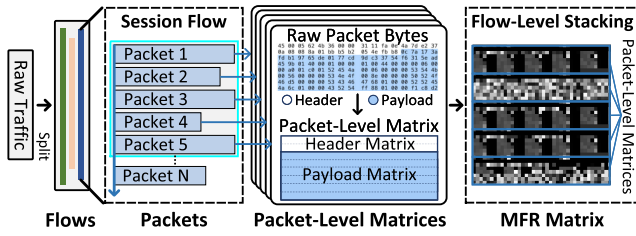


Fig. 1. The schematic illustration of multi-level flow representation (MFR). The raw traffic is decomposed hierarchically based on the structure, generating a two-dimensional representation matrix for each flow with three levels of traffic information: byte-level, packet-level, and flow-level.

a dedicated model structure for traffic classification based on traffic characteristics.

To address the challenges mentioned above, we introduce Yet Another Traffic Classifier (YaTC), a novel traffic classifier with self-supervised learning that learns latent representations from large amounts of unlabeled traffic data for more successful classification. Specifically, we first create a multi-level flow representation (MFR) matrix to model the raw traffic. It is generated from raw packet bytes and contains traffic information at different granularities through a formatted matrix, as illustrated in Fig. 1. Then, a novel Traffic Transformer with a packet-level attention module and flow-level attention module is constructed to analyze traffic using the MFR matrix. Finally, we train our classifier based on masked autoencoder (MAE) paradigm in two stages: pre-training and fine-tuning. In the pre-training stage, we randomly mask some packet bytes in the MFR matrix as input. Next, an autoencoder-based network, which consists of a traffic encoder and a small-scale decoder, reconstructs the original MFR matrix. Our traffic encoder leverages a large amount of unlabeled traffic data to learn latent representations through the reconstruction task. In the fine-tuning stage, we load the parameters of the pre-trained traffic encoder into Traffic Transformer and fine-tune it with a small number of labeled data for traffic classification. Our contributions can be briefly summarized as follows:

- We propose an MAE-based Traffic Transformer with MFR, called YaTC, for traffic classification. YaTC breaks through the traditional traffic analysis methods in three perspectives: traffic representation, classifier structure, and training strategy.
- We design an MFR matrix that fully considers the flow hierarchy to represent the raw traffic. To effectively utilize the MFR matrix for traffic analysis, we build a novel Traffic Transformer with packet-level and flow-level attention mechanisms. It can perform more efficient feature extraction with lower complexity and fewer parameters.
- We apply MAE-based self-supervised learning paradigm to train our classifier. It first leverages large-scale unlabeled traffic data to learn generic latent representations, and then performs fine-tuning with a small amount of labeled data for a series of traffic classification tasks.
- We evaluate YaTC on five real-world traffic datasets. Experimental results show that our method outperforms the state-of-the-art methods by a large margin. In addition, our pre-trained model exhibits excellent transfer ability for the new downstream traffic classification task.

The remainder of this paper is organized as follows. In Sec. II, we summarize the related work. Sec. III gives the preliminaries of our work. Sec. IV presents the design details of our proposed method. Sec. V reports extensive experiments. Sec. VI presents the detailed discussions of YaTC. Finally, we conclude this paper in Sec. VII.

II. RELATED WORK

A. Traffic Analysis Methods

1) *Rule-Based Methods*: In the early stage, researchers mainly used rule-based methods to accomplish traffic analysis. Through rules designed by security experts, basic attributes such as communication protocol and port number of traffic data are used to discover behaviors that violate security policies [31]. However, as network services and environments become more complicated, the fundamental features are insufficient to fulfill the current traffic analysis requirement [13].

2) *ML-Based Methods*: To analyze complicated traffic effectively, ML algorithms are introduced to explore the high-dimensional statistical features of traffic. For instance, CUMUL [12] selects 104 optimal statistical features through accuracy evaluation, which are then utilized as input for a support vector machine (SVM) to identify website traffic; AppScanner [13] adopts the random forest (RF) to analyze the statistical features generated by the traffic of different applications; [7] leverages ML algorithms to detect unknown patterns in encrypted malicious traffic by utilizing a compact in-memory graph; NetBeacon [32] directly deploys ML models on the network data plane for traffic analysis, improving model accuracy and deployment efficiency. Although ML-based methods combined with statistical features can analyze complex traffic, they rely heavily on statistical features designed by experts and need to select optimal features for different scenarios [14].

3) *DL-Based Methods*: DL-based approaches, which analyze traffic based on raw packets rather than human-designed features, have become the primary tool to automatically extract traffic representations and achieve remarkable performance improvement. In [26], a convolutional neural network (CNN) is used to identify traffic images for encrypted traffic classification. Reference [27] adopts the improved CNN classifier that leverages multiple channels to enrich feature information and improve analysis efficiency. TSCRNN [33] combines CNN and recurrent neural network (RNN) to learn the temporal characteristics from time-related packets. However, some existing DL-based methods directly use the packet bytes in the flow to represent the raw traffic, resulting in the bytes of a long packet overwhelming important information from other packets. Besides, they rely on sufficient labeled training data, and it is intractable to collect and manually label sufficient real traffic samples. Thus, we design an MFR matrix to represent multi-level information of the raw traffic through a formatted matrix and develop a novel Traffic Transformer to implement more efficient feature extraction based on these levels. To reduce the dependence on labeled data, we first leverage large-scale unlabeled data for pre-training, and then fine-tune with a few labeled data for traffic classification.

TABLE I
COMPARISON WITH STATE-OF-THE-ART METHODS

Method	Features	Model Structure	Task Type	Generality Evaluation	Pre-training	Params (10^4)
FlowPrint [23]	Statistical Features	Clustering	Data Mining	✗	N.A.	N.A.
AppScanner [13]	Statistical Features	Random Forest	Data Mining	✗	N.A.	N.A.
DF [24]	Direction Sequence	CNN	Computer Vision	✗	✗	145.4
Deeppacket [25]	Header + Payload	CNN	Computer Vision	✗	✗	4554.9
2D-CNN [26]	Header + Payload	CNN	Computer Vision	✗	✗	327.5
3D-CNN [27]	Header + Payload	CNN	Computer Vision	✗	✗	44.7
FS-Net [28]	Length Sequence	RNN	Computer Vision	✗	✗	105.7
CL-ETC [29]	Header + Payload	CNN	Computer Vision	✗	✓	682.5
PEAN [30]	Header + Payload	BERT	Natural Language Processing	✗	✓	218.9
PERT [21]	Payload	BERT	Natural Language Processing	✗	✓	5387.2
ET-BERT [22]	Payload	BERT	Natural Language Processing	✓	✓	13213.3
YaTC (Ours)	Header + Payload	Traffic Transformer	Computer Vision	✓	✓	186.2

B. Pre-Training Methods

Pre-training methods significantly reduce the appetite for labeled training data by self-supervised learning. Besides, unbiased data representations learned from large amounts of unlabeled data further improve performance on downstream tasks. Overall, the pre-training methods first obtain the pre-trained model with unlabeled data, and then load the model parameters to complete downstream tasks. In NLP tasks, BERT [19] is the most widely used to implement pre-trained models by cloze task and next sentence prediction. Some traffic classification studies [21], [22] consider the bytes of traffic packets as words and introduce BERT for pre-training to achieve better classification performance. However, since the input traffic bytes lack explicit high-level semantic units, latent representation extraction of traffic bytes is more suitable as a CV task than an NLP task. In CV tasks, traditional pre-training methods [16], [17] rely on data augmentation (e.g., cropping, enlarging) to form pairs of positive and negative samples for pre-training via contrastive learning. Obviously, data augmentation on the traffic image generated by the packet bytes will seriously damage the original information, making it difficult to apply the contrastive learning paradigm to our task. Benefiting from the introduction of the MAE [15], we are able to obtain an effective pre-trained model by masking patches for reconstruction training. Since the masking operation does not change other fixed-position packet bytes in the traffic image, it conforms to the design of fixed positions of different levels of packet content in multi-level flow representation.

TABLE I summarizes the most relevant studies for traffic classification, including their reference, classifier, features, the datasets used in each study, whether they use feature selection or not, and whether they perform generality evaluation on multiple datasets.

III. PRELIMINARIES

A. Problem Definition

We consider the problem of traffic classification as MAE-based traffic classifier learning. Our task is to obtain a generic pre-trained model with a large amount of unlabeled

traffic data, and then fine-tune this model for traffic classification with limited labeled traffic data. There are three datasets: an unlabeled training set, a labeled training set, and a test set. Note that the both training data sets and the test data set are disjoint. To verify the generality of our classifier, we comprehensively evaluate performance on multiple real-world traffic datasets generated by different types of networks (e.g., anonymous network, encrypted network, IoT network).

Raw traffic data is a collection of packets containing a sequence of raw bytes with varying granularity of information (e.g., header, payload, packet, and flow). The packet is raw byte data that contains raw byte data describing the header and payload. The flow is a group of packets with the same five-tuple (i.e., source/destination IP addresses, source/destination port numbers, and protocol). Obviously, these different information granularities reflect different characteristics of the traffic, so we design a multi-level flow representation including all aforementioned information granularities as the input of the classifier; see Section IV-A for details.

B. Transformer

Transformer [34] has strong sequence modeling capabilities and was first proposed to model sequences of words in NLP and achieved great success. Then, many works try to introduce Transformer into the CV field, among which Vision Transformer [35] is a typical paradigm.

Transformer is constructed by alternately stacking self-attention layers and feed-forward layers, where self-attention layer is the key to its powerful sequence modeling ability. Given the sequence input, the self-attention layer first calculates Query Matrix, Key Matrix, and Value matrix by using linear transformation, denoted as Q, K, V . Then the self-attention layer outputs the weighted sum of V according to the similarity between Q and K . The self-attention layer allows tokens in different positions to interact, while keeping the shape of the sequence output the same as the sequence input. The feed-forward layers perform a token-wise nonlinear transformation, which can strengthen the expressive ability of the sequence output.

Vision Transformer splits an image into a sequence of non-overlapping patches and then vanilla Transformer can be used to model sequences of patches. It has achieved SOTA results on many visual tasks. Since latent representation extraction of traffic packet bytes is more suitable as a CV task, we employ Vision Transformer as the backbone architecture. We modify the Vision Transformer to make it more suitable for extracting representations from raw packet bytes, denoted as Traffic Transformer, as detailed in IV-B. Although effective, self-attention is computationally expensive and its time complexity is $O(N^2)$, making it difficult for the vanilla Transformer to process long sequences. To reduce model complexity, we introduce the segmental attention strategy to reduce the input sequence length.

C. Masked Autoencoder

Masked Autoencoder (MAE) is a novel self-supervised learning paradigm specifically designed for Vision Transformer. Both the encoder and the decoder of MAE are Vision Transformer. The image is first divided into a sequence of non-overlapping patches. In the pre-training stage, some patches are randomly masked according to the masking ratio. By performing the pretext task of reconstructing masked patches from partially visible patches, the encoder is encouraged to extract high-quality representations from the patches and the decoder is able to reconstruct masked regions in pixel-level from visible representations. After pre-training, the encoder is preserved for downstream tasks.

IV. METHODOLOGY

In this section, we introduce our traffic representation method, classifier structure, and training strategy in detail in Section IV-A, Section IV-B, and Section IV-C, respectively. The overview of our YaTC is shown in Fig. 2.

A. Multi-Level Flow Representation

We design a novel method to produce multi-level flow representations from raw traffic data as input for traffic classification. Most existing methods directly intercept the preceding fixed number of bytes in the flow to form a two-dimensional matrix, which can be handled as an image and classified by DL algorithms. However, compared with the header, the size of the payload is usually significantly larger and is full of incomprehensible information generated by encrypt operation, resulting in excessive low-level semantic information in the matrix, which affects these models' effectiveness and efficiency. Moreover, in some flows, the first long packet will occupy the entire matrix, so the matrix cannot include the data in other packets of the traffic flow.

To address the above issues, we propose an MFR matrix with a formatted two-dimensional matrix to represent raw traffic flow, as illustrated in Fig. 1. The reason for representing the flow as a two-dimensional matrix instead of one-dimensional is to utilize the second dimension to portray multilevel information. First, we split the raw traffic into flows according to IP address, port number, and protocol type. Then, to avoid data

snooping or suspicious correlation, we remove the Ethernet headers of the flows, set the port numbers to zero, and replace the IPs with random addresses but keep their directions [36], [37]. This measure prevents the model from learning irrelevant information. Finally, the M adjacent packets in the flow are captured and formatted into a two-dimensional matrix of size $H * W$ as the representation of this flow, where H and W denote the height and width of the MFR respectively. Note that when there are lower than M packets in the flow, the MFR matrix only contains bytes of existing packets and other positions are not filled.

We still use the raw bytes as the initial characteristics of the traffic, i.e., the value of each point in the traffic representation matrix, but with the following particular design for capturing the multi-level information of the raw traffic.

- **Byte-level.** Each row of the traffic representation matrix includes only one type of traffic bytes, classified into header row and payload row.
- **Packet-level.** Each packet is represented by the header matrix and payload matrix, forming a packet-level matrix of size $H/M * W$.
- **Flow-level.** Since the flow is composed of ordered packets, M adjacent packet-level matrices are stacked in the second dimension to form the final MFR matrix.

In this way, the information at each level is fixed, and there will be no overflow at the lower level leading to the loss of information at the higher level. We set our MFR matrix to include 5 packet-level matrices with a total of 40 rows, and each row includes 40 bytes. Each packet's header is represented by 2 header rows with 80 bytes, with the capability to contain the IP layer header (20 bytes), TCP header (20 bytes) or UDP header (8 bytes), and optional headers. We assign 6 payload rows to accommodate the payload of each packet and perform truncation when it exceeds 240 bytes. Note that if the number of valid bytes is insufficient, it will be padded with 0 bytes to form a fixed-size representation matrix. We use this hierarchical design for traffic representation (fixed-position matrices to represent byte-level, packet-level, and flow-level information), which also makes it suitable to design a Transformer based classifier with position embedding for performance breakthroughs.

B. Traffic Transformer

According to the characteristics of traffic, we design our Traffic Transformer composed of an embedding module, a packet-level attention module, and a flow-level attention module, to effectively conduct traffic classification with the MFR matrix.

1) *Embedding Module:* The MFR matrix $x \in \mathcal{R}^{H \times W}$ is first split into non-overlapping 2D patches of size $P \times P$ with a total number of $N = HW/P^2$, denoted as $x_p \in \mathcal{R}^{N \times P^2}$, where P is the size of a patch. The patches are then mapped to D -dimensional vectors by a linear layer as patch embeddings. To maintain the position information, which is crucial in MFR, the position embeddings are added to the patch embeddings as the input of the traffic encoder.

$$x_0 = [x_p^1 E; x_p^2 E; \dots; x_p^N E] + E_{pos}. \quad (1)$$

By virtue of MFR's fixed positioning of packets, headers, and payloads, coupled with position embedding, our approach transcends the constraints of locality and can capture long-range dependencies within the traffic. Specifically, we take $D = 192$, $P = 2$ and $N = 20 * 20 = 400$, so that all elements of one patch represent the same type of raw bytes, and each packet matrix contains one row of header patches and three rows of payload patches.

2) *Packet-Level Attention Module*: In the packet-level attention phase, we develop a traffic encoder based on Vision Transformers [35], which consists of alternating multi-head self-attention layers (MSA) and feed-forward layers.

Aiming to preferentially learn the dependencies between header patches or payload patches within the packet, the traffic encoder only performs multi-headed self-attention among patches in the same packet rather than all patches in the MFR matrix. The MSA layer allows patches of the packet to interact with each other effectively according to the degree of relevance by different heads $Concat(head_1, \dots, head_n)$, and each head is computed by the attention function:

$$Q = x_l W^Q, K = x_l W^K, V = x_l W^V, \quad (2)$$

$$Attn(Q, K, V) = softmax(\frac{QK^T}{\sqrt{D_k}})V, \quad (3)$$

where $W^Q, W^K, W^V \in \mathcal{R}^{D \times D_k}$ are learnable parameters, $Attn$ denotes the attention function, and $1/D_k$ is the scaling factor. In this work, we employ $n = 16$ parallel attention heads and $L = 4$ alternating layers.

Note that our packet-level attention module focuses on promoting packet-level information interaction at this stage, which is also in line with the traffic characteristics that the information in the packet has a stronger correlation. In other words, it ignores the patch features of other packets and outputs a valid packet-level representation. Furthermore, our method also has lower time complexity compared to global attention ($O(N^2/M)$ vs. $O(N^2)$), which can benefit the efficiency of traffic analysis.

3) *Flow-Level Attention Module*: After packet-level attention, significant packet-level features of each patch of the MFR matrix $x'_p \in \mathcal{R}^{N \times D}$ have been output, which provides the basis for further use of the transformer to extract long-distance relational dependencies between different packets on the flow. At the flow level, it is unnecessary to continue with the same fine-grained patch as before, and even simply mean-pooling all the packet-level patch features can yield good performance. To learn inter-packet relationships at a coarser granularity, we employ row pooling (RP), which partially mean-pooling the output patches' features of the packet-level attention by row to generate row patches:

$$x_r = Pooling(x'_p), \quad (4)$$

where $x_r \in \mathcal{R}^{\sqrt{N} \times D}$ are row patches output of RP. Since each row of input patch features indicates the same type byte of traffic, the outputs of RP also can be divided into header or payload with fixed meaning.

In particular, each packet contains 1 header row patch and 3 payload row patches, representing the partial features within the packet and an MFR matrix has a total of $\sqrt{N}$ row patches.

We input all the row patches within an MFR matrix into a traffic encoder, and the MSA layers capture the flow-level information between the row patches. The final output is a column of row patch features $x_c \in \mathcal{R}^{\sqrt{N} \times D}$, which further perform column pooling (CP) to obtain the final representation $x_{MFR} \in \mathcal{R}^D$ of the entire MFR matrix:

$$x_{MFR} = Pooling(x_c). \quad (5)$$

Besides, with the coarser granularity, the model also has a lighter weight, the number of row patches is only $\sqrt{N}$ and the time complexity of this stage is only $O(N)$.

C. Training Strategy of YaTC

1) *Pre-Training YaTC*: As shown in Fig. 2, in the pre-training stage, YaTC utilizes MAE [15] paradigm with an asymmetric encoder-decoder architecture to reconstruct the raw bytes in the MFR matrix. A high percentage of the MFR patches are randomly masked, and only a tiny fraction of the patches (i.e. visible unmasked patches) are input to the model. After that, our traffic encoder extracts as many valid features of this fraction of the patches as possible, and then outputs the encoder token. Finally, a small decoder uses encoder token and mask token to recover the masked region of the MFR matrix. MAE is trained with a reconstruction loss, i.e., mean squared error (MSE) between the ground-truth pixels y_{real} and the predicted pixels y_{rec} :

$$\mathcal{L}_{rec} = MSE(y_{rec}, y_{real}). \quad (6)$$

After pre-training, the traffic encoder can extract high-quality features and is preserved for downstream tasks.

The pre-training does not require costly manual labeling, allowing the use of large amounts of unlabeled data from authentic scenes. Moreover, we find that a very high mask ratio (90%) achieves better results on downstream tasks. A high mask ratio also means that only a few patches are visible, resulting in lacking raw information to capture dependencies within and between packets. Thus, we perform global attention instead of packet-level attention and flow-level attention during pre-training. It avoids the introduction of structural knowledge and forces the traffic encoder to capture features on packet-level and flow-level together by reconstructing the corrupted information.

2) *Fine-Tuning YaTC*: On the downstream task, the encoder parameters from pre-training are loaded into both the packet-level attention module and flow-level attention module in the traffic transformer and used for feature extraction on packet-level and flow-level. For classification, the features of each patch are mean-pooled in two stages (RP and CP) to be used as the classification features of the MFR Matrix, which are flattened and input to an MLP to obtain the prediction distribution $\hat{y} \in \mathcal{R}^C$, where C is the number of traffic categories. Then classification loss is computed according to cross-entropy loss between prediction distribution $\hat{y}$ and the ground-truth label y :

$$\mathcal{L}_{CE} = H(\hat{y}, y). \quad (7)$$

To speed up convergence and reduce model size, inspired by ALBERT [20] and CYCLE [38], we share parameters between

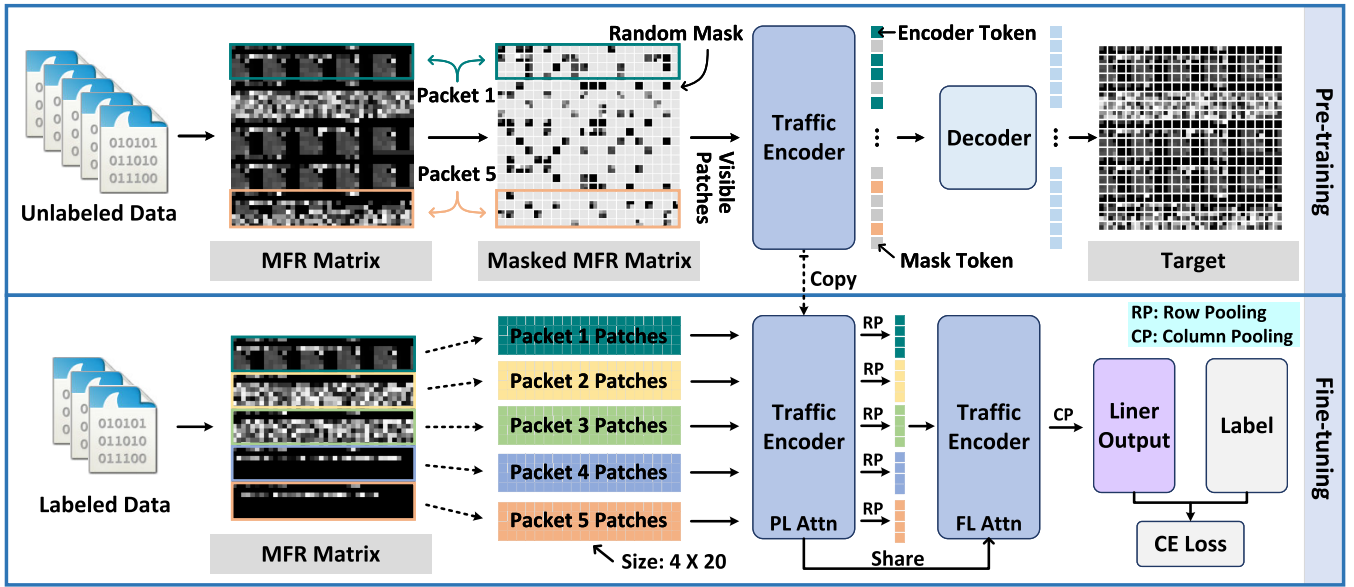


Fig. 2. The schematic illustration of YaTC. With MAE-based self-supervised learning paradigm, YaTC first learns generic latent representations from a large amount of unlabeled traffic data in the pre-training stage, then performs supervised learning with a few labeled data for a series of traffic classification tasks in the fine-tuning stage.

the two traffic encoders and achieve better performance with half the number of parameters. We argue the reason that both packet-level attention and flow-level attention are essentially performing inter-patch dependency capture, and the information interactions during multi-headed self-attention are similar. Moreover, the data in the same row of patches belong to the same type, i.e., header or payload, and the row-pooling's mean operation does not change the feature space in which the patches are located. So except for the difference in granularity, the two traffic encoders on the feature space are very similar to the stacking of transformer layers, which provides the conditions for the application of parameter sharing. In the fine-tuning stage, the two traffic encoders cause the double depth of Transformer layers compared to the pre-training stage, but the amount of training data is very limited. Thus, parameter sharing also makes the model easier to train.

V. EXPERIMENTS

A. Experiment Settings

1) *Data Preparation*: To verify the generality of our classifier, we comprehensively evaluate performance on five real-world traffic datasets generated by different types of networks (e.g., anonymous network, encrypted network, IoT network), as listed below:

- **ISCXVPN2016 dataset** [39] is used to evaluate the effectiveness of encrypted traffic classification on virtual private networks. The original dataset contains VPN scenario traffic and non-VPN scenario traffic. To reflect the performance of our method in VPN scenario, we use the VPN traffic with 7 categories for evaluation. It includes a total of 2,844 flows for classification. Besides, the selected dataset is divided into the independent training dataset and the test dataset according to the proportion of 80% and 20% for each class.
- **ISCTXor2016 dataset** [40] contains real-world traffic generated by the anonymous network and VPN network. The 8 categories of tor network traffic include chat, mail,

browsing, audio, video, FTP, VoIP, and P2P. The training dataset includes 1,456 flows and the test dataset includes 364 flows. According to the official description, the benign data coincides with the aforementioned ISCXVPN2016 dataset, so we only used tor traffic for evaluation.

- **USTC-TFC2016 dataset** [26] is a collection of real-world traffic consisting of ten categories of benign application traffic and ten categories of malware traffic. There are 3,900 flows in the training dataset and 982 flows in the test dataset.
- **CICIoT2022 dataset** [41] is the latest IoT traffic dataset for behavioral analysis and intrusion detection of IoT devices. We selected the traffic of 23 devices in the dataset for evaluation. The training dataset includes 7,457 flows and the test dataset includes 3,514 flows. There are seven benign traffic categories and three attack traffic categories in this dataset.
- **Cross-Platform dataset** [23] contains various application traffic under standard encryption protocols. We selected 59 categories of Android application traffic in the US region. There are 5,841 flows in the training dataset and 1,486 flows in the test dataset.
- **VPN datasets** [42] contains 3 UDP-based VPN traffic datasets generated by NordVPN, TurboVPN, and SuperVPN clients with 150 categories respectively. There are 6,000 flows for training, and 1,500 flows for testing.

To verify the generality of the method, we conduct evaluations on the above six datasets. The first four training datasets form a large-scale unlabeled training dataset for pre-training. In the fine-tuning stage, our classifier uses six training datasets for supervised learning to complete the corresponding traffic classification task. Note that a generic pre-trained model is obtained in the pre-training stage, while the classifier training in the fine-tuning stage is performed separately. Furthermore, the training datasets of Cross-Platform and VPN are not part of the unlabeled dataset in the pre-training stage. Thus, the VPN dataset is used to evaluate the

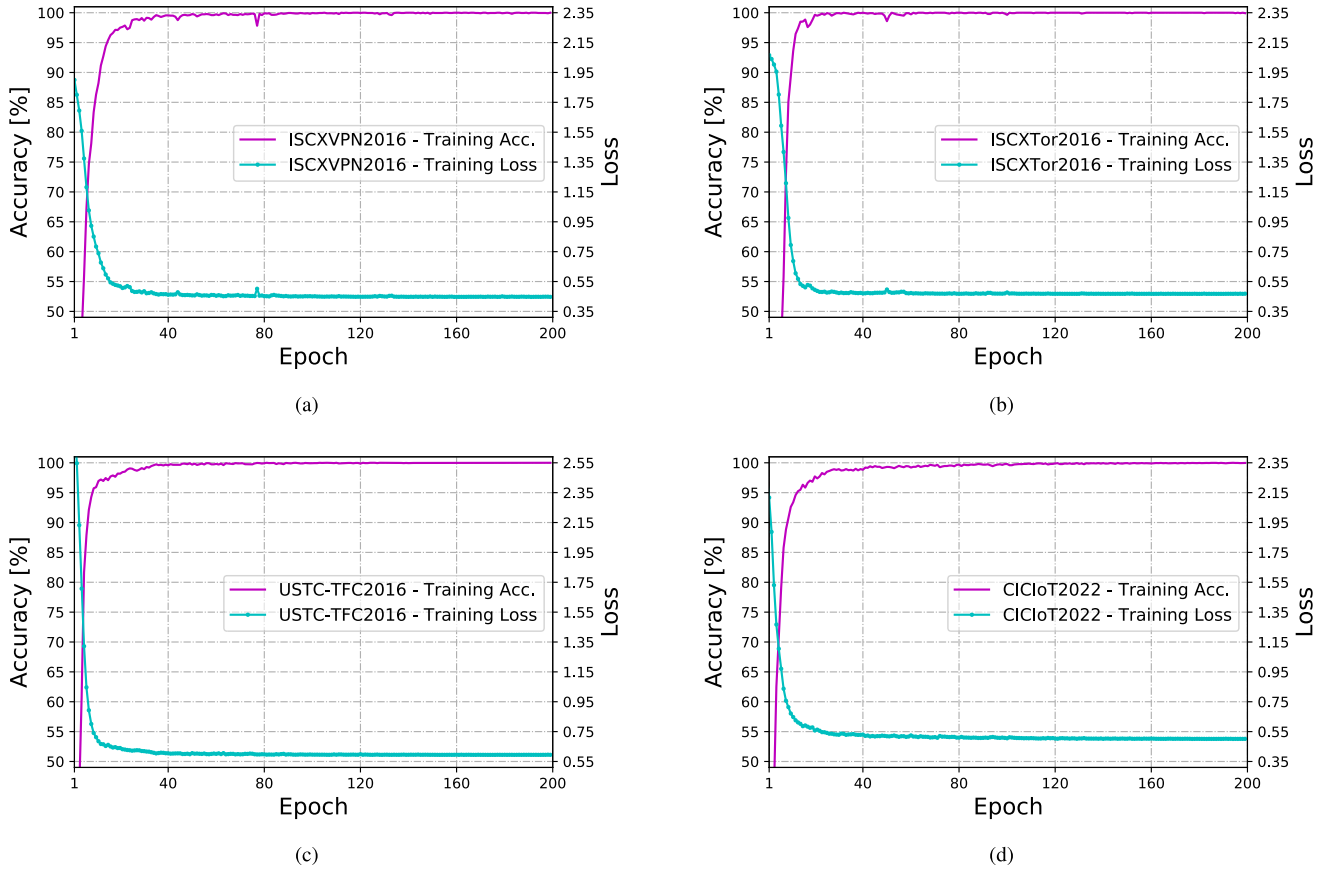


Fig. 3. The training accuracy and loss of YaTC on (a) ISCVPN2016 dataset, (b) ISCTor2016 dataset, (c) USTC-TFC2016 dataset, and (d) CICIOT2022 dataset.

model performance on new real-world traffic in Section V-E and the Cross-Platform dataset is used for transfer learning experiments in Section VI-B.

2) *Implementation Details*: In the pre-training phase, the batch size is 512 and the total steps is 150,000. We use the linear lr scaling rule and the base learning rate is set to 1×10^{-3} with AdamW optimizer. The masking ratio for randomly masking patches is set to 0.9. Then, we use the AdamW optimizer in fine-tuning for 200 epochs, where the base learning rate is set to 2×10^{-3} and batch size is 64. The impact of different masking ratios is discussed in Section VI. Besides, we strictly train each pre-training method using exactly the same data in our experiments. The proposed method is implemented using PyTorch 1.9.0 and trained on a server with four NVIDIA GeForce RTX3090 GPUs.

3) *Evaluation Metrics*: To measure the classification performance of our method, we calculate the number of True Positive (T_p), True Negative (T_n), False Positive (F_p), and False Negative (F_n). Based on the above definition, Recall, Precision and F_1 can be obtained:

$$Recall = \frac{T_p}{T_p + F_n}, \quad Precision = \frac{T_p}{T_p + F_p}, \quad (8)$$

$$F_1 = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall}. \quad (9)$$

B. Classification Performance of YaTC

As shown in Fig. 3, we plot the training accuracy and loss values of YaTC in the fine-tuning stage. Benefiting from

the excellent feature extraction ability, YaTC achieves very high training accuracy on the four training datasets. Besides, the variation of loss values shows that YaTC converges in 40 epochs, meaning that our method can efficiently complete the training tasks. After convergence, not only the performance improvement is limited, but also the increasing epochs bring unnecessary time and resource consumption. Thus, we stop model training at epoch 200.

Furthermore, we take a look at the classification performance of YaTC on the test datasets. As shown in Fig. 4, we make four confusion matrices to show the classification results performed by our method. Although the traffic classification tasks in the four datasets are different, YaTC still achieves very high accuracy in all scenarios, showing its ideal generality. Beside, we use micro average precision-recall (PR) curves to investigate the precision and recall of YaTC under different thresholds. It can be seen from Fig. 5 that our classifier makes a small number of false predictions with high confidence (i.e., using high thresholds) on the USTC-TFC2016 and CICIOT2022 datasets. We can conclude that YaTC can effectively perform traffic analysis tasks in different scenarios.

C. Comparison With State-of-the-Art Methods

To comprehensively evaluate the our method, we compare YaTC with a range of baselines and state-of-the-art methods, as listed below:

- **FlowPrint** [23] and **AppScanner** [13] are ML-based methods using statistical features for traffic classification.

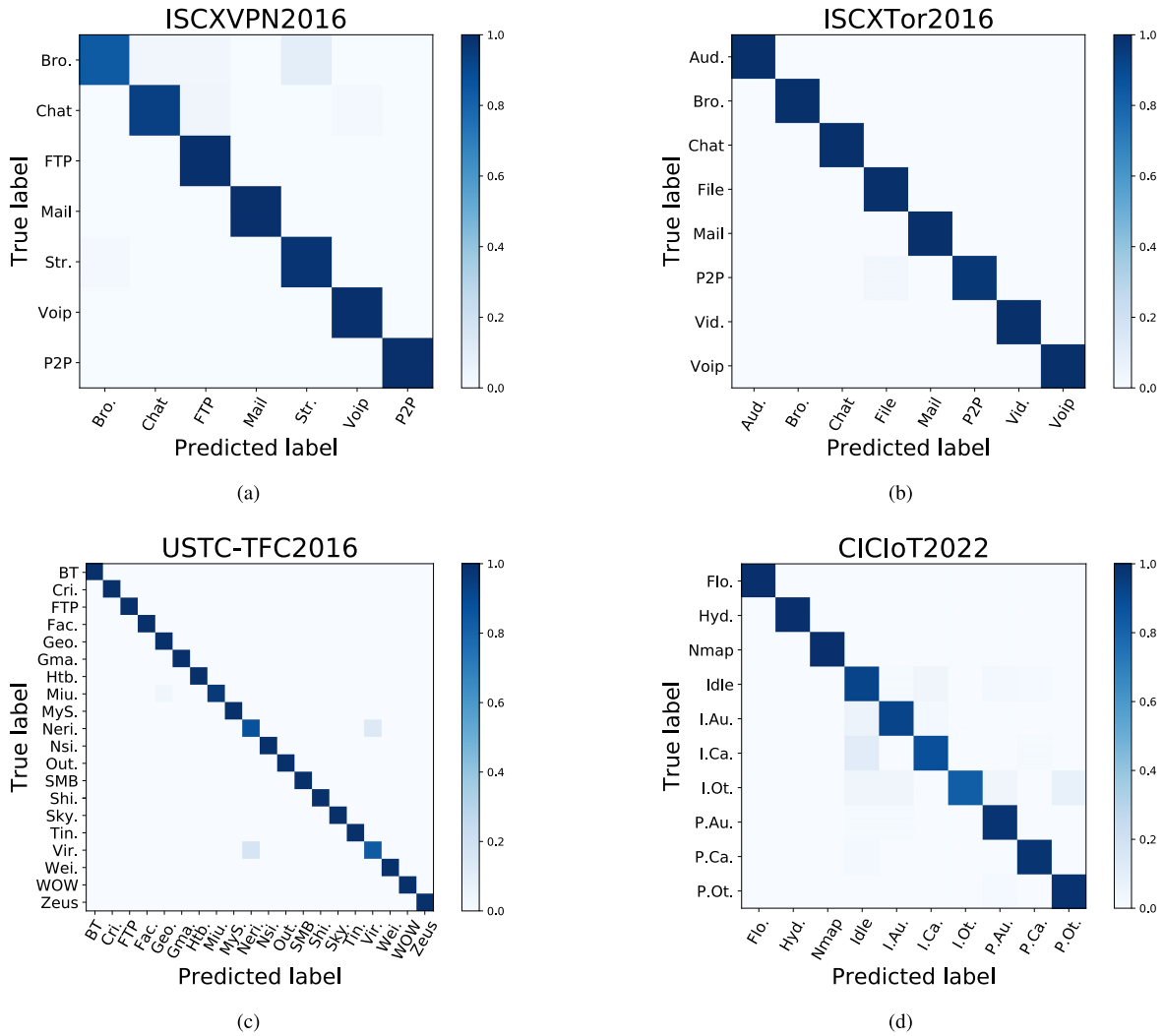


Fig. 4. Performance confusion matrices of YaTC on (a) ISCXVPN2016 dataset, (b) ISCTXor2016 dataset, (c) USTC-TFC2016 dataset, and (d) CICIOT2022 dataset.

TABLE II
COMPARISON RESULTS ON ISCXVPN2016, ISCTXor2016, USTC-TFC2016, AND CICIOT2022 DATASETS

Method	ISCXVPN2016		ISCTXor2016		USTC-TFC2016		CICIOT2022	
	Acc.	F_1	Acc.	F_1	Acc.	F_1	Acc.	F_1
FlowPrint	30.29%	14.09%	25.27%	10.19%	25.30%	12.47%	50.46%	49.14%
AppScanner	79.93%	80.85%	50.27%	49.68%	60.41%	58.36%	76.52%	76.81%
DF	62.87%	25.40%	33.24%	7.00%	58.45%	49.15%	60.13%	46.35%
Deeppacket	80.21%	80.17%	36.81%	26.81%	88.49%	88.83%	88.28%	88.08%
2D-CNN	81.26%	80.64%	34.62%	33.66%	92.26%	92.05%	90.07%	90.00%
3D-CNN	81.09%	80.79%	34.89%	33.96%	91.55%	91.16%	89.39%	89.33%
FS-Net	87.64%	87.30%	52.03%	51.64%	87.05%	86.02%	85.37%	85.30%
CL-ETC	80.91%	81.05%	41.48%	41.66%	94.20%	94.18%	93.80%	93.76%
PEAN	83.54%	82.71%	51.19%	44.92%	73.32%	67.91%	61.59%	63.63%
PERT	88.62%	88.61%	80.22%	79.99%	96.63%	96.64%	90.52%	90.49%
ET-BERT	87.74%	87.47%	65.38%	64.98%	96.95%	96.95%	90.35%	90.31%
YaTC (Ours)	98.07%	98.04%	99.72%	99.72%	97.86%	97.86%	96.58%	96.58%

- **DF** [24], **Deeppacket** [25], **2D-CNN** [26], **3D-CNN** [27], and **FS-Net** [28] are DL-based traffic analysis methods, which use raw packet information for supervised learning.
- **CL-ETC** [29], **PEAN** [30], **PERT** [21] and **ET-BERT** [22] treat traffic representation extraction as a CV or NLP task for pre-training, and then fine-tune the classifier with limited labeled data.

As is shown in TABLE II, the results demonstrate the effectiveness of DL-based methods and the insufficiency of ML-based methods with statistical features. Since only the vector of packets' directions in the flow is used as input features, the classification performance of DF is significantly lower than other DL-based methods. Furthermore, we observe that DL-based methods on the ISCTXor2016 dataset perform

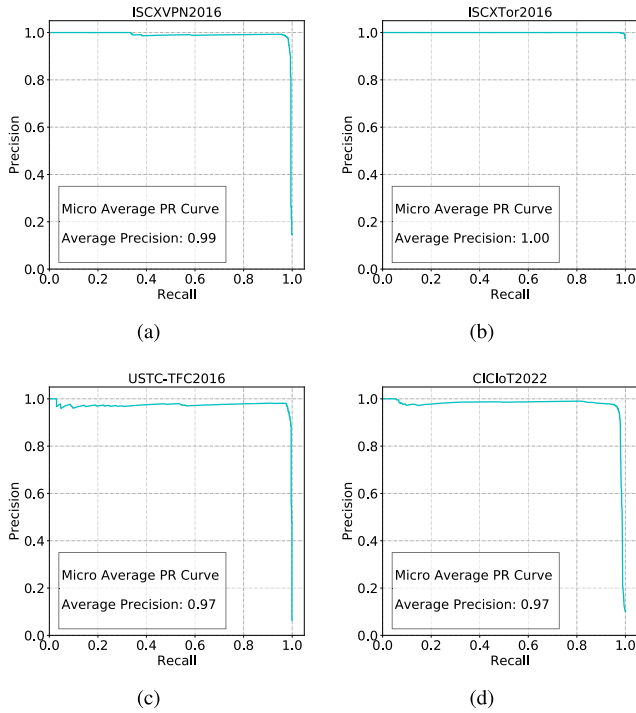


Fig. 5. The precision-recall curve of YaTC on (a) ISCXVPN2016 dataset, (b) ISCXTor2016 dataset, (c) USTC-TFC2016 dataset, and (d) CICIOT2022 dataset.

poorly, whether using sequence features or payloads as inputs. The reason is that encryption and obfuscation techniques for anonymous traffic make it difficult to analyze the payload directly, and the sequence features hardly work on limited captured packets. The characteristic of Tor traffic is the variable packet length with a highly obfuscated payload against traffic analysis. Thus, simply intercepting raw bytes would be full of incomprehensible information and unable to distinguish traffic information levels, but our MFR matrix for traffic representation can address this issue with hierarchical design. Besides, pre-training methods can learn latent representations from a large amount of unlabeled traffic data for more effective classification. Thus, PERT, ET-BERT, and YaTC outperform other methods on all datasets, illustrating that classifiers with the pre-training strategy can achieve better performance on downstream traffic classification tasks. However, not all pre-training methods can benefit from large-scale unlabeled data. We observe that CL-ETC and PEAN do not achieve obvious advantages over other supervised baselines. It illustrates the importance of the training strategy design as well as classifier structure. Benefiting from well-designed traffic representation, model structure, and training paradigm, YaTC consistently outperforms all compared methods by a large margin on all datasets. It can be concluded that our method is suitable for traffic analysis in various scenarios as well as has excellent performance.

To further study the effect of pre-training, we directly train the five classifiers without using pre-training. As shown in Fig. 6, the classification performance of PERT, ET-BERT, and YaTC using pre-training is significantly improved compared to that without pre-training. However, CL-ETC has weak performance due to its shallow pre-training model (two-layer CNN structure) and lack of feasible pre-training strategy.

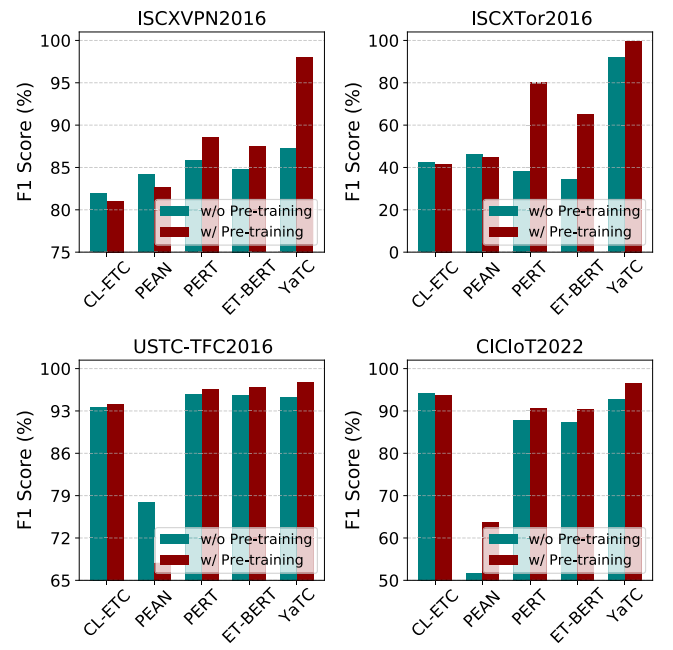


Fig. 6. The performance improvements using pre-trained models.

PEAN only pre-trains part of the modules in the model, so the pre-training cannot contribute to the entire traffic analysis model, making it inferior to other three Transformer-based methods. Furthermore, we observe that YaTC achieves the best performance with or without pre-training compared to the other two methods on ISCXVPN2016, ISCXTor2016, and CICIOT2022 datasets, indicating that our MAE-based pre-training paradigm and model structure are more effective and robust.

D. Ablation Study

To further examine the level of benefit that each component of YaTC brings to the performance, an ablation study is performed on four baseline datasets. The evaluation results are reported in TABLE III. First, compared with applying global attention, our method not only reduces the complexity (discussed in Section IV-B.2) but also achieves better results, demonstrating the benefits of packet-level and flow-level attention mechanisms. Furthermore, removing packet-level attention leads to significant performance degradation under all configurations, which illustrates the considerable contribution of packet-level feature extraction. We note that flow-level attention without pre-training sometimes leads to performance loss, which would not occur after pre-training. This is because further attention to small data without pre-training is easier to cause over-fitting. The ablations also show that applying parameter sharing during fine-tuning brings both lightweight and performance enhancements. Moreover, directly transforming the raw traffic bytes without MFR or removing only the flow-level stacking in MFR also results in weaker performance, obviously for the encrypted traffic tasks.

Furthermore, we ablate the header part and payload part of the MFR matrix separately. It can be seen from Fig. 7 that removing the header brings a more obvious performance drop than removing the payload on the four datasets, illustrating that

TABLE III
ABLATION STUDY OF KEY COMPONENTS IN YATC ON ISCXVPN2016, ISCXTor2016, USTC-TFC2016, AND CICIOt2022 DATASETS

Method	ISCXVPN2016		ISCXTor2016		USTC-TFC2016		CICIOt2022	
	Acc.	F_1	Acc.	F_1	Acc.	F_1	Acc.	F_1
YaTC (Ours)	98.07%	98.04%	99.72%	99.72%	97.86%	97.86%	96.58%	96.58%
with GA	95.27%	95.14%	98.63%	98.62%	97.86%	97.86%	95.64%	95.61%
w/o PA	90.19%	90.03%	78.02%	77.28%	96.03%	96.03%	92.84%	92.78%
w/o FA	95.62%	95.49%	99.18%	99.18%	97.66%	97.62%	95.81%	95.80%
w/o FS	92.47%	92.35%	97.80%	97.77%	93.48%	93.48%	94.58%	94.57%
w/o PS	97.55%	97.53%	99.45%	99.45%	97.45%	97.40%	95.41%	95.39%
w/o PT	87.74%	87.22%	92.03%	91.90%	95.32%	95.25%	92.70%	92.65%
w/o PT & PA	78.63%	77.58%	39.84%	38.58%	93.28%	93.22%	90.88%	90.79%
w/o PT & FA	87.74%	87.40%	85.99%	85.84%	95.52%	95.46%	93.19%	93.17%
w/o PT & FS	81.96%	81.84%	83.52%	83.15%	91.75%	91.48%	91.59%	91.59%
w/o PT & MFR	80.91%	80.49%	42.86%	42.11%	93.99%	93.90%	91.36%	91.26%

The abbreviations are explained as follows, GA: global attention, PA: packet-level attention, FA: flow-level attention, FS: flow-level stacking, PS: parameter sharing, MFR: multi-level traffic representation, and PT: pre-training.

TABLE IV
COMPARISON RESULTS AND ABLATION STUDY ON NORDVPN, TURBOVPN, AND SUPERVPN DATASETS

Method	NordVPN		TurboVPN		SuperVPN	
	Acc.	F_1	Acc.	F_1	Acc.	F_1
CL-ETC	8.33%	8.10%	19.53%	19.57%	33.60%	34.38%
PEAN	3.79%	0.71%	1.53%	0.39%	1.80%	0.56%
PERT	82.40%	82.25%	88.60%	88.18%	87.67%	87.36%
ET-BERT	63.07%	62.90%	87.87%	87.59%	83.67%	83.41%
YaTC (Ours)	92.07%	91.95%	96.47%	96.45%	98.33%	98.32%
w/o PT	74.00%	73.77%	86.07%	85.61%	77.87%	77.58%
w/o header	53.80%	53.40%	96.13%	96.12%	97.27%	97.26%
w/o payload	19.13%	16.83%	36.80%	34.89%	41.67%	40.27%

the header information is more important for traffic analysis. However, YaTC is still able to perform traffic classification using only the payload, and even achieves an F_1 score of 83.70% on the encrypted traffic dataset ISCXVPN2016. Previous work has analyzed the imperfect randomness of encrypted traffic payloads [22], which supports the rationale for us to retain encrypted payloads in our MFR. The experimental results show that using the header part and the payload part can jointly boost classification performance.

E. Performance on New Real-World Traffic

In real-world cases, the model needs to analyze various traffic data that is unrelated to the pre-training data. Thus, we evaluate the generalization of our method on new real-world traffic data with entirely new VPN protocols (i.e., WireGuard, IKEv2, and PPTP). The results are shown in TABLE IV. We observe that YaTC's classification performance is far ahead of other baselines. It shows that YaTC maintains high feasibility even in the face of the traffic data with new protocols. PERT and ET-BERT achieve sub-optimal performance, indicating that their advanced paradigms are suitable for fine-tuning and analysis in new environments. In contrast, CL-ETC and PEAN perform not very well due to the lack of feasible pre-training paradigms. Further, it can be seen that pre-training significantly boosts the classification performance of YaTC in this scenario. It also proves that our pre-trained model is very critical to the generalization of the

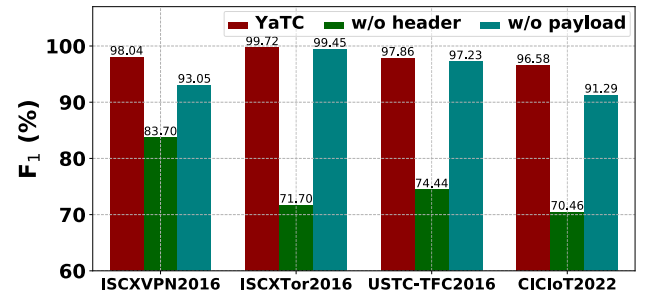


Fig. 7. The ablations on the header part and the payload part of our MFR matrix.

method. In addition, we perform ablation experiments on the header and payload of traffic data respectively. Results indicate header is less important than the payload when analyzing these new VPN traffic. This is because they apply entirely new VPN protocols, which are mainly reflected in the header, making it less effective. The payload contributes significantly to classifying these VPN traffic data.

F. Effectiveness of MFR Matrix for Traffic Representation

1) *Header Matrix Analysis*: At the lowest level, the protocol header bytes feature clear semantic information and have different lengths according to the protocol. It is essential to maintain the completeness of header bytes. Concretely, we do not use the MAC layer header, which contains only the upper layer protocol type, mac addresses, and checksums. Instead, we read the packet's header starting from the IP layer. Each packet's header is represented by two header rows totaling 80 bytes, with the capability to contain the IP layer header (20 bytes), TCP header (20 bytes) or UDP header (8 bytes), and optional header segments that may exist. When the number of header bytes is less than 80, we fill in zero at the end of the header to ensure the integrity and purity of the header row.

2) *Payload Matrix Analysis*: For payload bytes, its length varies significantly between packets. Sometimes the packet doesn't contain a payload, and sometimes it is easy to exceed the expected number even though the maximum transmission unit (MTU) limits it. The payloads are usually fragmented and have no completeness anymore. However, they can still be classified by deep learning methods, which indicates that

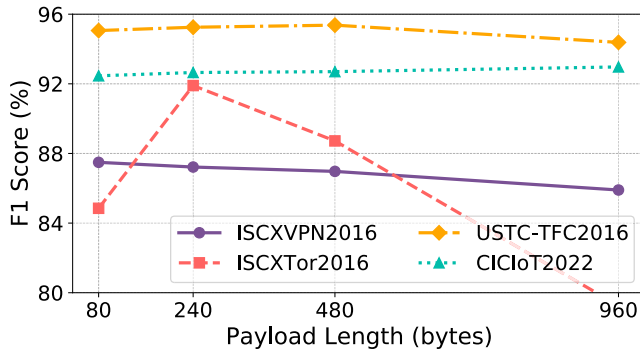


Fig. 8. The performance comparison using different payload lengths.

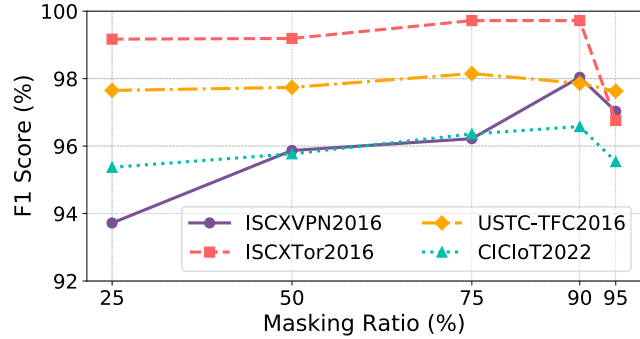


Fig. 9. The impact of masking ratio on the quality of learned representation.

the integrity of a payload is not required for classification, and a portion of the payload can reveal its feature. Thus, we assign 6 payload rows to each packet to accommodate possible payloads, and intercept the payload when it exceeds 240 bytes; otherwise, fill the padding with zero. As shown in Fig. 8, we conduct experiments to analyze the impact of different payload lengths. To reduce interference factors, we adopt supervised learning paradigm (i.e., without pre-training) to train our classifier with the fixed header length (80 bytes) and different payload lengths, avoiding the impact of pre-trained models on performance. We can observe that on the Tor dataset, intercepting the payload length of 240 bytes can achieve the optimal classification performance, which is significantly higher than other payload lengths. On the other three datasets, different payload lengths have no significant effect on the classification performance. In addition, a longer payload length also brings an increase in model complexity. Thus, we set 240 bytes as the payload length in our MFR matrix for excellent generality as well as low complexity.

VI. DISCUSSIONS

A. Impact of Masking Ratio

To explore the effect of the masking ratio on the quality of learned representation, we conduct experiments on four datasets with a range of masking ratios. Experimental results are shown in Fig. 9. Overall, a higher mask ratio will result in better performance. On the other hand, an excessively high masking ratio makes the reconstruction task too difficult, and the F_1 score decreases rapidly. The F_1 score reaches the highest when the masking ratio is 90% on most datasets except USTC-TFC2016, which also has a high best masking ratio of 75%. This supports our view of the traffic task as being more

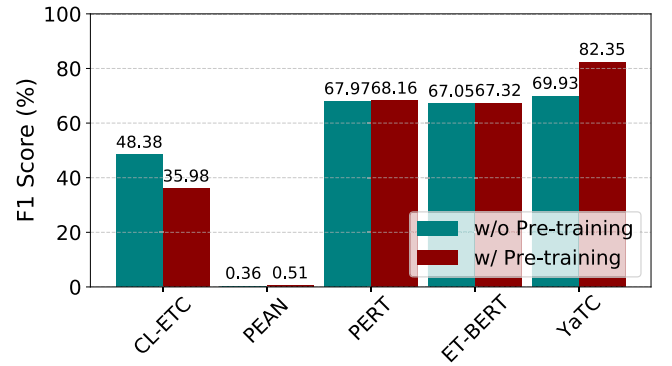


Fig. 10. The performance comparison with the other two pre-training methods on the Cross-Platform dataset.

similar to images than to words. In the masking strategy of NLP, such as BERT, the optimal masking ratio maintains a low value and usually does not exceed 20%, which is the opposite of the CV according to the recent study [15]. A high optimal masking rate implies a large information redundancy, and traffic classification does not require a thorough understanding of its content, which is also impossible on the encrypted payload. For classification tasks, words are high-level and abstract information, but traffic bytes are just sparse features without clear semantic units, similar to pixels. Hence, rather than the BERT-based methods treating raw bytes as words, our image-like MFR matrix is more rational.

B. Transfer Learning Experiments

We evaluate transfer learning on the Cross-Platform dataset. Note that the fine-tuned and test data of this dataset is unrelated to pre-training. Four other pre-training traffic classification methods (CL-ETC, PEAN, PERT and ET-BERT) are introduced for comparison. The pre-trained models are all identical to those used in the previous experiments (i.e., pre-trained with the other four datasets). It can be seen from Fig. 10 that our YaTC significantly improves the F_1 score from 69.93% to 82.35%, which is 12.42% better than without pre-training. Furthermore, both ET-BERT and PERT have weak boosts using pre-training, indicating that their pre-trained models are difficult to transfer to the new downstream traffic classification task. Neither the CL-ETC nor PEAN pre-training paradigms contribute to enhance their performance in this scenario, which is consistent with the previous analysis in Section V-C.

C. Few-Shot Analysis

To validate the robustness of YaTC in few-shot scenarios, we set the labeled data size to 10%, 50%, and 100% and compare YaTC with other baselines and state-of-the-art methods on four datasets. Experimental results are shown in Fig. 11. Experiments show that the three pre-training-based methods, YaTC, ET-BERT, and PERT, generally outperform other supervised methods in few-shot scenarios. Some other methods achieve similar performance to pre-training-based methods in a few cases, but none of them could maintain it on all four datasets. The pre-training enables the model to master the ability to extract high-quality representations using unlabeled data, thereby greatly reducing the dependence on labeled data. Besides, YaTC achieves better performance than

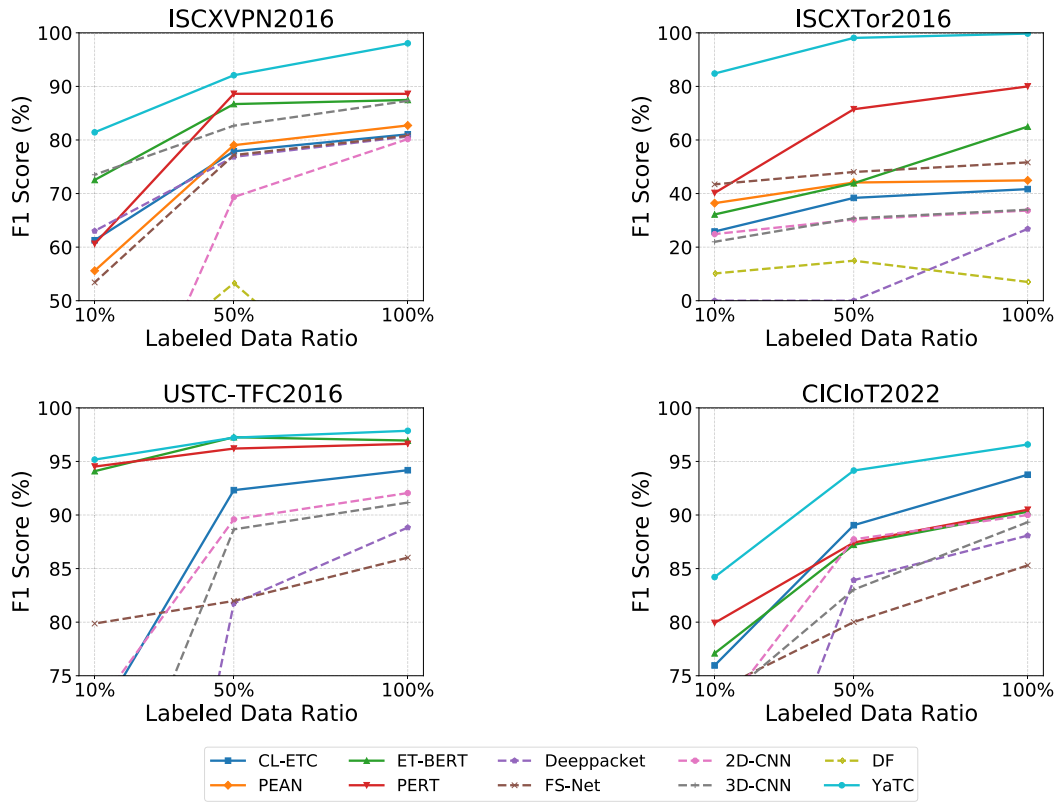


Fig. 11. The performance comparison with other models using different ratios of labeled data.

TABLE V
THE EFFICIENCY OF YATC AND PRE-TRAINING BASELINES

Method	Inference Time (ms)		Throughput (samples/s)	
	GPU	CPU	GPU	CPU
CL-ETC	0.9012	14.8094	71015.61	4321.59
PEAN	81.9346	2606.3872	585.83	18.42
PERT	45.8977	715.3850	697.20	44.73
ET-BERT	45.9399	729.1206	696.56	43.89
YaTC	21.8508	244.3353	2928.95	261.94

ET-BERET and PERT at all sizes of labeled data, indicating excellent robustness.

D. Deployment Mode

Our well-trained model can be used to analyze the MFR matrix at any time, so the deployment mode of YaTC is determined by the generation strategy of the MFR matrix. With offline mode, there is sufficient time to analyze a large amount of collected traffic data. The traffic is first divided into session flows, then M ordered packets of one flow are intercepted to generate the MFR matrix, and finally classified by our model. With online mode, YaTC will satisfy the real-time requirement by setting a time window (e.g., 1s, 10s, and 20s) to collect several consecutive packets that are on the same flow to generate the MFR matrix, and then classify M ordered packets online. In future work, we will conduct in-depth research on deployment to improve the scalability of our method.

E. Efficiency of YaTC

Since timeliness is crucial for traffic analysis, we further analyze the efficiency of YaTC and pre-training baselines.

TABLE V shows that the average inference time per sample of our method on our platform is approximately 22 ms in GPU mode and 244 ms in CPU mode. The throughput of YaTC is approximately 2929 samples/s in GPU mode and 262 samples/s in CPU mode. We observe that CL-ETC achieves extremely high efficiency. The reason is that it only uses a two-layer CNN structure as the backbone, which also results in low classification performance and weak effectiveness of pre-training. Furthermore, the efficiency of the other three Transformer-based pre-training methods is significantly lower than ours. It can be concluded that YaTC can perform traffic analysis tasks efficiently.

VII. CONCLUSION

In this work, we proposed YaTC, an MAE-based Traffic Transformer with MFR for traffic classification. With the MAE-based self-supervised learning paradigm, YaTC first learns generic latent representations from a large amount of unlabeled traffic data in the pre-training stage, then performs supervised learning with a few labeled data for a series of traffic classification tasks in the fine-tuning stage. Our method is evaluated on five real-world traffic datasets. Experimental results show that our YaTC is ahead of the state-of-the-art methods, even in the case of minimal labeled data. Besides, our pre-trained model exhibits excellent transfer ability for the new downstream traffic classification task. Although YaTC has high efficiency compared to other Transformer-based methods, we believe that the analysis efficiency can be further improved. For example, pruning technology can be introduced to achieve better lightweight performance. In future work, we will conduct research on these deployment-related issues.

REFERENCES

- [1] R. Zhao et al., "Yet another traffic classifier: A masked autoencoder based traffic transformer with multi-level flow representation," in *Proc. AAAI Conf. Artif. Intell. (AAAI)*, 2023, pp. 1–8.
- [2] E. Papadogiannaki and S. Ioannidis, "A survey on encrypted network traffic analysis applications, techniques, and countermeasures," *ACM Comput. Surv.*, vol. 54, no. 6, pp. 1–35, Jul. 2022.
- [3] R. Zhao, X. Deng, Z. Yan, J. Ma, Z. Xue, and Y. Wang, "MT-FlowFormer: A semi-supervised flow transformer for encrypted traffic classification," in *Proc. 28th ACM SIGKDD Conf. Knowl. Discovery Data Mining*, Aug. 2022, pp. 2576–2584.
- [4] K. Xie, R. Xie, X. Wang, G. Xie, D. Zhang, and J. Wen, "NMMF-stream: A fast and accurate stream-processing scheme for network monitoring data recovery," in *Proc. IEEE Conf. Comput. Commun. (IEEE INFOCOM)*, May 2022, pp. 2218–2227.
- [5] J. Fan, C. Guan, K. Ren, and C. Qiao, "Middlebox-based packet-level redundancy elimination over encrypted network traffic," *IEEE/ACM Trans. Netw.*, vol. 26, no. 4, pp. 1742–1753, Aug. 2018.
- [6] C. Fu, Q. Li, M. Shen, and K. Xu, "Frequency domain feature based robust malicious traffic detection," *IEEE/ACM Trans. Netw.*, vol. 31, no. 1, pp. 452–467, Feb. 2023.
- [7] C. Fu, Q. Li, and K. Xu, "Detecting unknown encrypted malicious traffic in real time via flow interaction graph analysis," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2023, pp. 1–18.
- [8] L. Yang, A. Moubayed, and A. Shami, "MTH-IDS: A multitiered hybrid intrusion detection system for Internet of Vehicles," *IEEE Internet Things J.*, vol. 9, no. 1, pp. 616–632, Jan. 2022.
- [9] P. Dodia, M. AlSabah, O. Alrawi, and T. Wang, "Exposing the rat in the tunnel: Using traffic analysis for tor-based malware detection," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Nov. 2022, pp. 875–889.
- [10] M. Shen et al., "Machine learning-powered encrypted network traffic analysis: A comprehensive survey," *IEEE Commun. Surveys Tuts.*, vol. 25, no. 1, pp. 791–824, 1st Quart., 2023.
- [11] Z. Cai, B. Jiang, Z. Lu, J. Liu, and P. Ma, "IsAnon: Flow-based anonymity network traffic identification using extreme gradient boosting," in *Proc. Int. Joint Conf. Neural Netw. (IJCNN)*, Jul. 2019, pp. 1–8.
- [12] A. Panchenko et al., "Website fingerprinting at internet scale," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2016, pp. 1–15.
- [13] V. F. Taylor, R. Spolaor, M. Conti, and I. Martinovic, "AppScanner: Automatic fingerprinting of smartphone apps from encrypted network traffic," in *Proc. IEEE Eur. Symp. Secur. Privacy (EuroSP)*, Mar. 2016, pp. 439–454.
- [14] M. Shen, Y. Liu, L. Zhu, K. Xu, X. Du, and N. Guizani, "Optimizing feature selection for efficient encrypted traffic classification: A systematic approach," *IEEE Netw.*, vol. 34, no. 4, pp. 20–27, Jul. 2020.
- [15] K. He, X. Chen, S. Xie, Y. Li, P. Dollár, and R. Girshick, "Masked autoencoders are scalable vision learners," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2022, pp. 1–10.
- [16] M. Caron et al., "Emerging properties in self-supervised vision transformers," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2021, pp. 9630–9640.
- [17] X. Chen, S. Xie, and K. He, "An empirical study of training self-supervised vision transformers," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2021, pp. 9620–9629.
- [18] R. Zhao et al., "GeeSolver: A generic, efficient, and effortless solver with self-supervised learning for breaking text captchas," in *Proc. 44th IEEE Symp. Secur. Privacy (SP)*, May 2023, pp. 1–18.
- [19] J. Devlin, M. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. North Amer. Chapter Assoc. Comput. Linguistics, Hum. Lang. Technol. (NAACL-HLT)*, 2019, pp. 4171–4186.
- [20] Z. Lan, M. Chen, S. Goodman, K. Gimpel, P. Sharma, and R. Soricut, "ALBERT: A lite BERT for self-supervised learning of language representations," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2020, pp. 1–17.
- [21] H. Y. He, Z. Guo Yang, and X. N. Chen, "PERT: Payload encoding representation from transformer for encrypted traffic classification," in *Proc. ITU Kaleidoscope, Ind.-Driven Digit. Transformation (ITU K)*, Dec. 2020, pp. 1–8.
- [22] X. Lin, G. Xiong, G. Gou, Z. Li, J. Shi, and J. Yu, "ET-BERT: A contextualized datagram representation with pre-training transformers for encrypted traffic classification," in *Proc. ACM Web Conf.*, Apr. 2022, pp. 633–642.
- [23] T. van Ede et al., "FlowPrint: Semi-supervised mobile-app fingerprinting on encrypted network traffic," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2020, pp. 1–18.
- [24] P. Sirinam, M. Imani, M. Juárez, and M. Wright, "Deep fingerprinting: Undermining website fingerprinting defenses with deep learning," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Oct. 2018, pp. 1928–1943.
- [25] M. Lotfollahi, M. J. Siavoshani, R. S. H. Zade, and M. Saberian, "Deep packet: A novel approach for encrypted traffic classification using deep learning," *Soft Comput.*, vol. 24, no. 3, pp. 1999–2012, Feb. 2020.
- [26] W. Wang, M. Zhu, X. Zeng, X. Ye, and Y. Sheng, "Malware traffic classification using convolutional neural network for representation learning," in *Proc. Int. Conf. Inf. Netw. (ICOIN)*, Jan. 2017, pp. 712–717.
- [27] J. Zhang, F. Li, F. Ye, and H. Wu, "Autonomous unknown-application filtering and labeling for DL-based traffic classifier update," in *Proc. IEEE Conf. Comput. Commun. (IEEE INFOCOM)*, Jul. 2020, pp. 397–405.
- [28] C. Liu, L. He, G. Xiong, Z. Cao, and Z. Li, "FS-Net: A flow sequence network for encrypted traffic classification," in *Proc. IEEE Conf. Comput. Commun. (IEEE INFOCOM)*, Apr. 2019, pp. 1171–1179.
- [29] Z. Zhao, Y. Guo, J. H. Wang, H. Wang, C. Zhang, and C. An, "CL-ETC: A contrastive learning method for encrypted traffic classification," in *Proc. IFIP Netw. Conf. (IFIP Networking)*, Jun. 2022, pp. 1–9.
- [30] P. Lin, K. Ye, Y. Hu, Y. Lin, and C.-Z. Xu, "A novel multimodal deep learning framework for encrypted traffic classification," *IEEE/ACM Trans. Netw.*, vol. 31, no. 3, pp. 1369–1384, Jun. 2023.
- [31] T. T. T. Nguyen and G. Armitage, "A survey of techniques for internet traffic classification using machine learning," *IEEE Commun. Surveys Tuts.*, vol. 10, no. 4, pp. 56–76, 4th Quart., 2008.
- [32] G. Zhou, Z. Liu, C. Fu, Q. Li, and K. Xu, "An efficient design of intelligent network data plane," in *Proc. 32nd USENIX Sec. Symp.*, 2023, pp. 1–18.
- [33] K. Lin, X. Xu, and H. Gao, "TSCRNN: A novel classification scheme of encrypted traffic based on flow spatiotemporal features for efficient management of IIoT," *Comput. Netw.*, vol. 190, May 2021, Art. no. 107974.
- [34] A. Vaswani et al., "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst. (NIPS)*, 2017, pp. 5998–6008.
- [35] A. Dosovitskiy et al., "An image is worth 16×16 words: Transformers for image recognition at scale," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2021, pp. 1–22.
- [36] D. Arp et al., "Dos and don'ts of machine learning in computer security," in *Proc. 31st USENIX Sec. Symp.*, 2022, pp. 3971–3988.
- [37] A. S. Jacobs, R. Beltiukov, W. Willinger, R. A. Ferreira, A. Gupta, and L. Z. Granville, "AI/ML for network security: The emperor has no clothes," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Nov. 2022, pp. 1537–1551.
- [38] S. Takase and S. Kiyono, "Lessons on parameter sharing across layers in transformers," 2021, *arXiv:2104.06022*.
- [39] G. Draper-Gil, A. H. Lashkari, M. S. I. Mamun, and A. A. Ghorbani, "Characterization of encrypted and VPN traffic using time-related features," in *Proc. 2nd Int. Conf. Inf. Syst. Secur. Privacy*, 2016, pp. 407–414.
- [40] A. H. Lashkari, G. D. Gil, M. S. I. Mamun, and A. A. Ghorbani, "Characterization of tor traffic using time based features," in *Proc. 3rd Int. Conf. Inf. Syst. Secur. Privacy*, 2017, pp. 253–262.
- [41] S. Dadkhah, H. Mahdikhani, P. K. Danso, A. Zohourian, K. A. Truong, and A. A. Ghorbani, "Towards the development of a realistic multidimensional IoT profiling dataset," in *Proc. 19th Annu. Int. Conf. Privacy, Secur. Trust (PST)*, Aug. 2022, pp. 1–11.
- [42] S. Oh, M. Lee, H. Lee, E. Bertino, and H. Kim, "AppSniffer: Towards robust mobile app fingerprinting against VPN," in *Proc. ACM Web Conf.*, Apr. 2023, pp. 2318–2328.



Ruijie Zhao (Graduate Student Member, IEEE) received the M.S. degree from Shanghai Jiao Tong University, Shanghai, China, in 2021, where he is currently pursuing the Ph.D. degree with the School of Electronic Information and Electrical Engineering. His research interests include machine learning, network security, and traffic analysis.



Mingwei Zhan received the B.E. degree in computer science and technology from Xi'an Jiaotong University, Xi'an, China, in 2021. He is currently pursuing the M.S. degree with the School of Electronic Information and Electrical Engineering, Shanghai Jiao Tong University, Shanghai, China. His research interests include deep learning, network traffic analysis, and network security.



Xianwen Deng (Student Member, IEEE) received the B.Eng. degree from Shanghai Jiao Tong University, Shanghai, China, in 2021, where he is currently pursuing the Ph.D. degree with the School of Electronic Information and Electrical Engineering. His current research interests include captcha systems, network traffic analysis, and deep learning.



Fangqi Li (Graduate Student Member, IEEE) received the M.S. degree in cyber science and engineering from Shanghai Jiao Tong University, Shanghai, China, in 2022, where he is currently pursuing the Ph.D. degree in cyber science and engineering with the School of Electronic Information and Electrical Engineering. His research interests include the security of machine learning systems and their applications in computer security.



Yanhao Wang received the Ph.D. degree in computer applied technology from the Chinese Academy of Sciences in 2019. His research interests are software security, network security, and the IoT security.



Yijun Wang received the B.S. and M.S. degrees from Shanghai Jiao Tong University, Shanghai, China, in 2001 and 2004, respectively. He is currently an Engineer with the School of Electronic Information and Electrical Engineering, Shanghai Jiao Tong University. His research interests include system security, network security, and machine learning.



Guan Gui (Senior Member, IEEE) received the Ph.D. degree from the University of Electronic Science and Technology of China, Chengdu, China, in 2012. From 2009 to 2014, he joined the Tohoku University as a Research Assistant and a Post-Doctoral Research Fellow, respectively. From 2014 to 2015, he was an Assistant Professor with Akita Prefectural University, Akita, Japan. Since 2015, he has been a Professor with the Nanjing University of Posts and Telecommunications, Nanjing, China. He has published more than 200 IEEE journals/conference papers and won several best paper awards, e.g., ICC 2017, ICC 2014, and VTC 2014-Spring. His recent research interests include artificial intelligence, deep learning, non-orthogonal multiple access, wireless power transfer, and physical layer security.

He received the IEEE Communications Society Heinrich Hertz Award in 2021, a member and Global Activities Contributions Award in 2018, the Top Editor Award of IEEE TRANSACTIONS ON VEHICULAR TECHNOLOGY in 2019, the Outstanding Journal Service Award of *KSII Transactions on Internet and Information System* in 2020, the Exemplary Reviewer Award of IEEE COMMUNICATIONS LETTERS in 2017, the 2012 Japan Society for Promotion of Science (JSPS) Post-Doctoral Fellowships for Foreign Researchers, and the 2018 Japan Society for Promotion of Science (JSPS) International Fellowships for Overseas Researchers. He was also selected as for the Jiangsu Specially-Appointed Professor in 2016, the Jiangsu High-Level Innovation and Entrepreneurial Talent in 2016, the Jiangsu Six Top Talent in 2018, and the Nanjing Youth Award in 2018. He was recognized as one of the 2020 highly cited Chinese researchers in wireless communications. He is serving or served on the editorial boards for several journals, including IEEE TRANSACTIONS ON VEHICULAR TECHNOLOGY, *IEICE Transactions on Communications, Physical Communication, Wireless Networks*, IEEE ACCESS, *Journal of Circuits, Systems, and Computers, Security and Communication Networks, IEICE Communications Express, KSII Transactions on Internet and Information Systems*, and *Journal of Communications*. In addition, he served as the IEEE VTS Ad Hoc Committee Member for AI Wireless, the Executive Chair for VTC 2021-Fall, the Vice Chair for WCNC 2021, the TPC Chair for PHM 2021, the Symposium Chair for WCSP 2021, the General Co-Chair for Mobimedia 2020, the TPC Chair for WiMob 2020, the Track Chair for EuCNC 2021, VTC 2020 Spring, ISNCC 2020, and ICC 2020, the Award Chair for PIMRC 2019, and a TPC Member for many IEEE international conferences, including GLOBECOM, ICC, WCNC, PIMRC, VTC, and SPAWC.



Zhi Xue received the B.S. and Ph.D. degrees from Shanghai Jiao Tong University, Shanghai, China, in 1992 and 2001, respectively. He is currently a Professor with the School of Electronic Information and Electrical Engineering, Shanghai Jiao Tong University. He was the Leader of more than 30 state/province projects of China and authored or coauthored more than 60 papers. His research interests include machine learning, network security, and data science.